

; Comments are preceded by the ';' sign. This line is a comment (is ignored by the assembler)

bits 32 ; assembling for the 32 bits architecture

global start ; we ask the assembler to give global visibility to the symbol called start
;(the start label will be the entry point in the program)

extern exit ; we inform the assembler that the exit symbol is foreign
;it exists even if we won't be defining it

import exit msvcrt.dll ;we specify the external library that defines the symbol
; msvcrt.dll contains exit, printf and all the other important C-runtime functions

; our variables are declared here (the segment is called data)

segment data use32 class=data

a dw 3

b dw 4

; the program code will be part of a segment called code

segment code use32 class=code

start:

mov ax, [a] ; ax = a

add ax, [b] ; ax = a + b = 7

; call exit(0) , 0 represents status code: SUCCESS

push dword 0 ; saves on stack the parameter of the function exit

call [exit] ; function exit is called in order to end the execution of
the program

- Byte 0-7
- Word 0-15
- Doubleword 0-31 (Low Word / High Word)
- Quadword 0-63 (Low Double Word / -4-)

constants IA-32

- numbers
 - base 2 : 101b, 11100b
 - base 16 : 34ABh, 0ABCdh
 - base 10 : 20, -114

• character

• string

Declaring constant (assembly language)

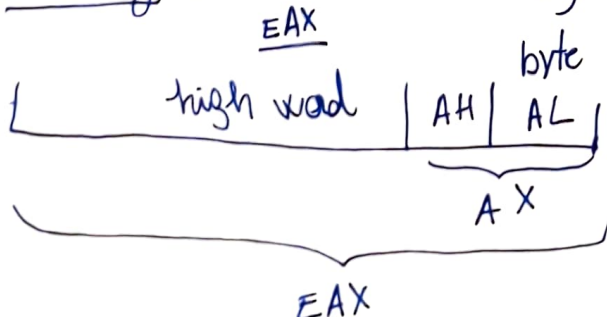
ten EQU 10

→ 2 kinds of variables

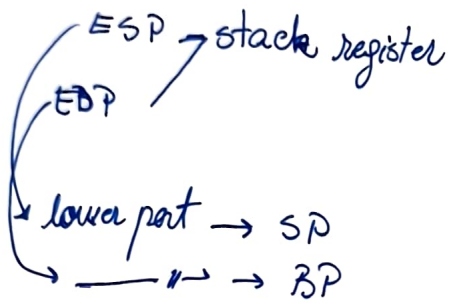
- predefined var (registers)
 - general (32 bits/register)
 - segment (16 bits/register)
 - other (32)

FLAG → sets what the register does

General reg. (EAX, EBX, ECX, EDX)



L - lower byte
H - higher byte



EDI - index register → DI

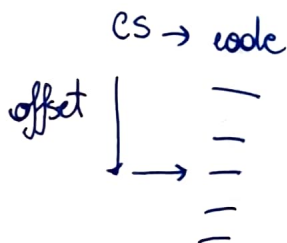
2. Segment registers

- CS - code
 - DS - Data
 - SS - stack
 - ES - Extra
- segment

↓ storing addresses

3. Other reg

- EIP & Flags
- ↓
- IP (subregister)



User def var

name, data, type, initial value

name byte value
a D 13 2 3

↓
declare byte

a DW 23ABh base 16

↓
declare define word

a 12 13 101

↓
a



User def var

a RESB 1 $\rightarrow$ reserve 1 byte

a RESB 64 $\rightarrow$ reserve 64 bytes

a RESW

MOV instruction

$\rightarrow$ copy a value from a.

~~dest = s~~ dest & source = registers, var, const

dest $\neq$ constant

Effect dest = source

Restrictions: both operands $\rightarrow$ same size

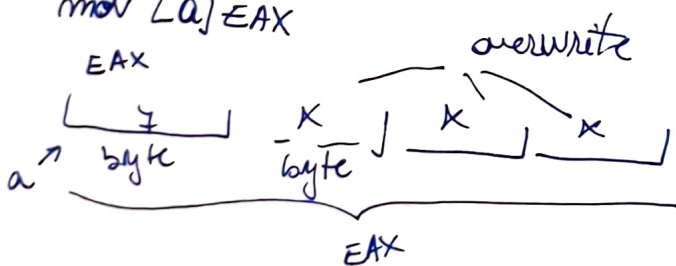
Ex: mov ax, 3; ax = 3

mov bx, ax; bx = ax

mov [a], eax

Ex: a db 7

mov [a], eax



ADD instruction

Syntax: ADD dest, source

Effect: dest = dest + source

at least 1 = register/constant

Functions

parameters $\rightarrow$ dwords

add [a], 3 $\rightarrow$ wrong (type!) ^{data}

add dword [a], 3 $\rightarrow$ right

SUB instruction

Syntax: SUB dest, source

Effect: dest = dest - source

Repr of integer numbers in the computer's memory

CPU $\rightarrow$ repr. an integer ~~number~~ ^{number} on 1, 2, 4, 8 bytes on the IA-32

unsigned $\text{repr}(\oplus \text{numbers}) = \text{repr of that number in base 2}$
_{positive}

signed representation of numbers

- bit sign + normal representation of the number

ex: $5 = 0000\ 0101$

$-5 = 1000\ 0101$

- we don't use a complement of 1 $\rightarrow$ invert all bits

ex: $5 = 0000\ 0101$

$-5 = 1111\ 1010$ } we don't get a 0

- the signed repr. of a positive number is equal to the unsigned repr. of that number in base 2

—— 11 — neg number = repr. in 2's complementary

ex: 17 on 8 bits

$0001\ 0001$
 $\downarrow$
positive

ex: -17

$$\begin{array}{r} 1\ 0000\ 0000 - \\ \underline{1\ 0001} \\ 1\ 1110\ 1111 \end{array}$$

(hexa)
 $10 \xrightarrow{-1} = 16 - 1 = 15 \text{ (F in hexa)}$
 $\underline{10}$
 F

17: $0001\ 0001$
 $1110\ 1111$

(copy)
We stop till we found
the first 1
then invert 0 $\leftrightarrow$ 1

exercises

1. Write a program that computes $4+2$

~~mov ax~~

bits 32

global start

extern exit

import exit msvcrt.dll

segment data use32 class = ~~code~~ data

i no variables needed, but we can use them

segment code use 32 class = code

start :

~~mov ax, 4~~ = values

mov ax, 4 → al, but they are small values

add ax, 2

push dword 0

call [exit]

2. mov ax, 1
sub ax, 5

1-5

4. Write a program that computes $(a+b)-c$, a, b, c - doubleword

bits 32

global start

extern exit

import exit msvcrt.dll

segment data use32 class data

a dd 1

b dd 2

c dd 3

segment code use32 class = code

start

mov eax, [a]

add eax, [b]

sub eax, [c]

push dword

call [exit]

-39

1 0000 0000 0000 0000
10 0111
1111 1111 1101 1001

filled with 1/0 (pos/neg) for bigger numbers

0001 0001 0100 1000
1110 1110 + 1011 1000
1
1110 1111

Interpretation

mul instruction → multiplies the value from AL reg with the value from the BL reg and store the result in AX

- signed
- unsigned > interpretation

AL = 11101111

- unsigned: positive (classic) $1 * 2^7 + 1 * 2$
- signed
 - 1 - negative
 - complementary code

AL = 00000101 unsigned: 5
= 5 signed: still 5 (positive)

signed & unsigned instructions

mov, add, sub → don't care abt. signed/unsigned

do care: div mul (unsigned)
: idiv, imul (signed)

repr. of numbers.

-10 on a word

1010 - binary

~~0110~~

10: 0000 1010 $\rightarrow$ -10: 1111 1111 0110

1111 1111 1111 0110

$$10:2 = 5 \text{ R } 0$$

$$5:2 = 2 \text{ R } 1$$

$$2:2 = 1 \text{ R } 0$$

$$1:2 = 0 \text{ R } 1$$

-21 on a word

~~0000 0000 0000 0000~~ ~~00101~~ ~~01011~~ $\rightarrow$ 10100
100
-21:

[0, 255]

[-128, 127]

$a \in [-300, 100] \rightarrow$ data type: word

$$\begin{array}{r} 21:2 = 10 \text{ R } 1 \\ \hline 20 \end{array}$$

$$10:2 = 5 \text{ R } 0$$

$$5:2 = 2 \text{ R } 1$$

$$2:2 = 1 \text{ R } 0$$

$$1:2 = 0 \text{ R } 1$$

$$7 = 8 + 4 + 2 + 1$$

$$2^1 \quad 3 \quad 2 \quad 1 \quad 0$$

$$A = 8 + 2$$

$$1 \quad 0 \quad 1 \quad 0$$

$$1 \rightarrow 0001$$

$$4 \rightarrow 0111$$

$$\begin{array}{r} 10+ \\ -10 \\ \hline 0 \end{array}$$

Base 16 to base 2

$$BCB1_{(16)} = 0110 \ 1100 \ 1011 \ 0001_{(2)}$$

$$10 \rightarrow 0000 \ 1010 \Rightarrow 0Ah$$

B_2 representation in memory

→ negative numbers

$$0000 \ 1010 +$$

[?]

$$1 \ 0000 \ 0000$$

reversed digit (extra)

representations (no losing memory) complementary code

$$\begin{array}{r} 1 \ 0000 \ 0000 \\ - \\ 0000 \ 1010 \\ \hline 11110110 (-10) \end{array}$$

ex: -10

$$100 - (16)$$

$$0A$$

$$\begin{array}{r} 100 \\ -0A \\ \hline F6 \end{array}$$

$$\downarrow \rightarrow (16-10)$$

$$\begin{array}{r} 0000 \ 1010 \\ \downarrow \\ 1111 \ 0110 \end{array} \rightarrow \text{last 2 the same}$$

$$1111 \ 0110_{(2)} \rightarrow 128 + 64 + 32 + 16 + 4 + 2 = 246_{(10)}$$

$$-10$$

H/x/

$$\begin{array}{r} 1010 - (2) \\ 1 \\ \hline 1001 \end{array}$$

$$\begin{array}{r} 01 \\ 1000 - (12) \\ 10 \\ \hline 110 \end{array}$$

$$\begin{array}{r} 6. \quad 7+2 \\ 7+ (16) \\ 2 \\ \hline 15 \end{array}$$

$$\begin{array}{r} (16) \\ 10+ \rightarrow 16 \\ A \\ \hline 1A \\ 26 \rightarrow 1A \\ 26/16 \\ 16/1 \\ \hline 10 \end{array}$$

Signed and unsigned instructions (multiplications, divisions, conversions)

$$11111111h * 11111111h \rightarrow 255 \cdot 255$$

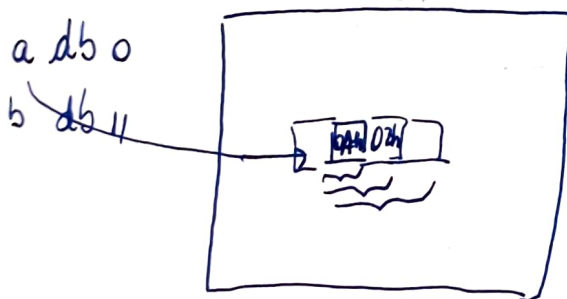
$$\rightarrow (-1) \cdot (-1)$$

unsigned: div, mul

signed: idiv, imul

ADD [a], [b]; ! incorrect

ADD [a], 1; ! incorrect, we are dividing

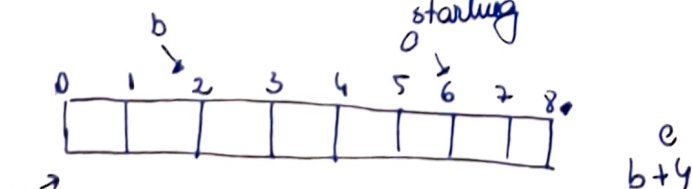


⇒ ADD byte [a], 1

ADD [a], AL

mov EAX, [a] ; moves the double word

mov EAX, a ; moves the address
starting



a dw 0

b dd 1

c dw 2

mul

source → byte ⇒ AX = AL * source

source → word ⇒ DX:AX = AX * source

source → dword ⇒ EDX:EAX = EAX * source

DX : AX
dw dw
high low
notation

EDX : EAX
dword dw
high low

div

source = byte $\Rightarrow AL = AX / \text{source}$

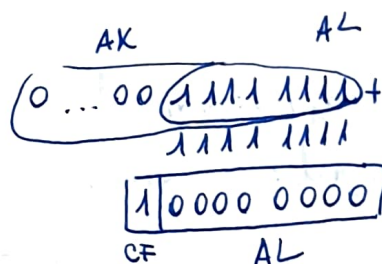
$AH = AX \% \text{source}$

source = word $\Rightarrow$

① $x = \overset{\text{byte}}{(a+b) * c} / d$

UNSIGNED + a, b, c, d - bytes

DATA SEGMENT	CODE SEGMENT
a db 10	mov AL, [a]
b db 11	add AL, [b]
c db 12	mul mul byte [c]; $AX = (a+b) * c$
d db 13	div byte [d]; $AL = ((a+b) * c) / d$ $AH = ((a+b) * c) \% d$
x db 0	mov [x], AL
(X RESB 1)	



AL $\rightarrow$ AX : mov AH, 0

BX $\rightarrow$ CX : BX : mov CX, 0

AX $\rightarrow$ EAX : mov BX, AX

mov EAX, 0

mov AX, BX

EAX $\xrightarrow{\text{unsigned}}$ EDX : EAX

mov EDX, 0

DX : AX $\xrightarrow{\text{unsigned}}$ EDX : EAX

PUSH DX ; DX = 1234h

PUSH AX ; AX = 5678h

POP EAX (high DX, low AX)

mov EDX, 0

idiv bx; $AX = \frac{a \cdot b}{d}$

mov [x], AX

~~sub [x], [c]~~

mov AL, [c]

cbw; AL → AX

sub (x), AX

④ $x + \frac{a \cdot b}{c \cdot d} + f \cdot g$

SIGNED + x - quad, a, d - byte, c - double, b, f, g - word

DATA SEGMENT	CODE SEGMENT
a db 10	mov AL, [a]
b dw 11	cbw; AX = a
c dd 12	imul word [b]; DX:AX = a * b
d db 13	push DX; high part
f dw -14	push AX; low part
g dw 15	pop EAX
x dq 0	cdq; EDX:EAX = a * b
	mov EBX, EAX
	mov ECX, EDX; ECX:EBX = a * b
mov ECX, [x+4]	mov AL, [d]
mov EBX, [x]	cbw
add EBX, EAX	cwde; EAX = d
adc ECX, EDX; ECX:EDX = x + $\frac{a \cdot b}{c \cdot d}$	mov EDX, [c]
mov AX, [f]	sub EDX, EAX; EDX = c - d
imul word [g]; DX:AX = f * g	xchg EDX, ECX; EDX:EBX = a * b
push EDX	ECX = c - d
push AX	mov EAX, EBX; EAX
pop EAX	idiv ECX; EAX = (a * b) / (c - d)
cdq; EDX:EAX = f * g	cdq; EDX:EAX = (a * b) / (c - d)
add EAX, EBX	
adc EDX, ECX	

Signed conversions

CBW ; AL $\rightarrow$ AX

CWD ; AX $\rightarrow$ DX:AX

CQW ; AX $\rightarrow$ EAX

CQD ; EAX $\rightarrow$ EDX:EAX

(2)

$$x = (a - b * c) / d$$

b * c - word

a - byte $\rightarrow$ word

UNSIGNED + a, b, c, d - byte

DATA SEGMENT	CODE SEGMENT	<u>SIGNED</u>
a db 10	mov AL, [b]	✓
b db 11	mul byte [c] ; AX = b * c	imul byte [c]
c db 12	mov BL, [a]	mov BX, AX
d db 13	mov BH, 0 ; BX = a	mov AL, [a]
x db 1	sub BX, AX	CBW
	mov AX, BX	sub AX, BX
	div byte [d]	✓
	mov [x], AL	idiv
		✓

(3) $x = \frac{a * b}{d} - 1$

SIGNED + a, c, d - byte b - word

DATA SEGMENT	CODE SEGMENT	
a db	mov AL, [a]	a - byte $\rightarrow$ word
b dw	CBW ; AX = a	a * b $\rightarrow$ d word
c db	imul word [c] ; DX:AX = a * b - byte $\rightarrow$ word	d - byte $\rightarrow$ word
d db	mov BX, AX ; DX:BX = a	$\frac{a * b}{d} - \text{word}$
	mov AL, [d]	X - word
	CBW ; AX = d	
	xchg AX, BX ; swap DX:AX = a * b DX = d	

$EDX: EAX +$
 $ECX: EBX$

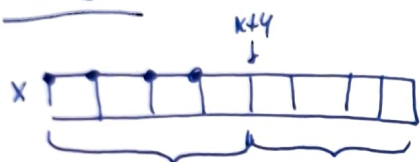
 $EBX + ECX \neq EAX + EBX$

$ADD EAX, EBX$
 $ADC EDX, ECX$

→ push/pop on the same

$DX: AX +$

$CX: BX$



dword low dword high

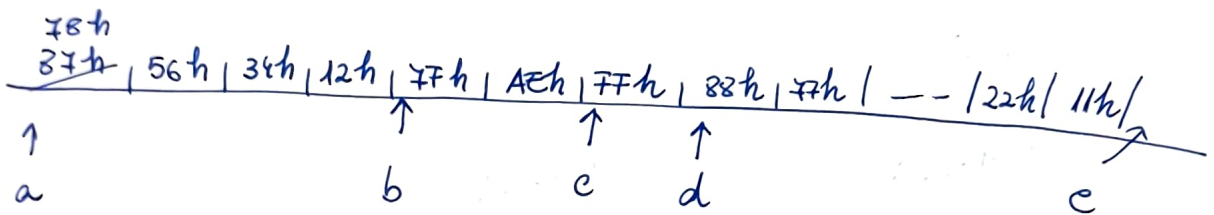
$mov ECX, [x+4]$

$mov EBX, [x]$

ASC - Seminar 3

03.11.2022

Little Endian representation



a dd 12345678h

b dw 0AE7Fh

c db 0FFh

d dq 1122334455667788h

e dw 20

20 $\rightarrow$ 10000

-20 = FFECh

AX = 88FFh

~~78h 56h 34h 12h 7Fh~~

MOV AX, [a]

MOV DX, [a+2]

low byte is the first one in the data segment

① address high byte of b =

• value b =

② value of the high byte of a

= 34h

• address of the low byte of the high word a

③ adr of the high byte of the low word of the high dword

$a * b + c * d$, a-byte, b, c, d-word

a db 2

b dw 3

c dw 4

d dw 5

~~mov AL, [a]~~

~~mov AH, 0~~

~~mul ~~word~~ [b]~~

~~mov BX, ~~AX~~ [b]~~

~~mov AX, [c]~~

~~mul word [d]~~

mov AL, [a]

mov AH, 0

mov BX, [b]

mul BX; res $\rightarrow$ DX:AX

mov BX, AX

mov BX, DX

mov AX, [c]

~~mov BX, [d]~~

mul ~~word~~ [d]; res $\rightarrow$ DX:AX = $c * d$

add AX, BX

adc DX, CX

mov AL, [a]

mov AH, 0

mov BX, [b]

mul BX

push DX

push AX

mov AX, [c]; res $\rightarrow$ DX:AX = $c * d$

mov BX, [d]

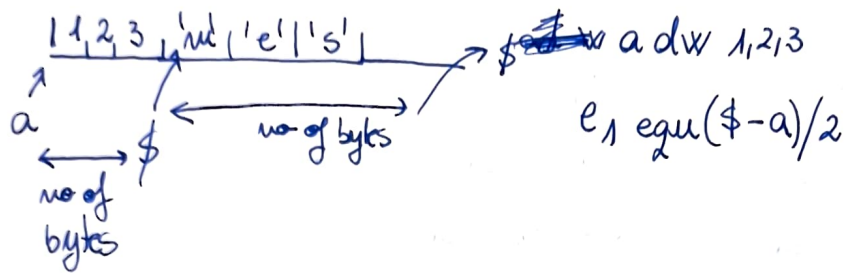
mul BX

pop BX $\leftarrow$ CX:BX $\rightarrow$ AX:BX

pop CX $\leftarrow$ $\left. \begin{array}{l} \text{add AX, BX} \\ \text{adc DX, CX} \end{array} \right\}$

\$ - location counter

Strings



→ declare len as a constant

'm', 'e', 's', ... (=) 'message'

Address = address of the 1st element

Length = no of elements

Direction for iterating the string

Ex: a) ESI - index of the string

`addb 'abcdef'`

`l1 equ $ - a`

`resb l1`

`mov ESI, 0`

`cmp ESI ESI, l1`

`JB instr`

`redo`

`cmp ESI, l1`

`JB instr`

`JMP fin`

instr:

`mov [b + ESI]`

`mov AL, [a + ESI]`

`sub AL, 'a' - 'A'`

`mov [b + ESI], AL`

`inc ESI`

`JMP redo`

fin:

`push dword 0`

`call [exit]`

2) ECX: EBX - EAX

a) UNSIGNED

mov EDX, 0

sub EBX, EAX

sbb ECX, EDX

b) SIGNED

cdq; EAX → EDX: EAX

sub EBX, EAX

sbb ECX, EDX

! EDX: EAX - EBX

{ push EDX

{ push EAX

mov EAX, EBX

~~cdq~~ mov EDX, 0

mov EBX, EAX

mov ECX, EDX

} pop EAX

} pop EDX

sub EAX, EDX

sbb EDX, ECX

JNA label 1

jump if not above

⋮

label 1

a - b

a < b (a below b) OF =

if (C1)

{ --- }

else

{ --- }

cmp -- C1

J l1 (true)

l1: if

else jmp l2

b) ESI - offset of the current byte

code segment:

mov ESI, a

mov EDI, b

redo:

~~cmp~~

mov ECX, 2

redo:

jecx: end

mov AL, [ESI]

sub AL, 'a' - 'A'

mov(EDI), AL

inc ESI

inc EDI

(dec ECX)

(jump redo) LOOP redo

end:

push dword 0

call [exit]

ASC - Seminar

17.11.2022

$$24:16 = 1 \text{ zu } 8$$
$$116 = 0 \lambda 1$$

Show the data segment for the following variables

a, db '24', '68' - 12.

a_2 dw 24, 68 -18

a3 d 2,4,6,8

a4 dw 2,4,6,8

a5 db 2468h

a6 dw 24h, 68h

a7 dd 02040608h

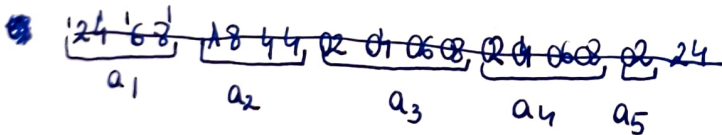
Q8 db 24h, 68h

ag dw 2468h

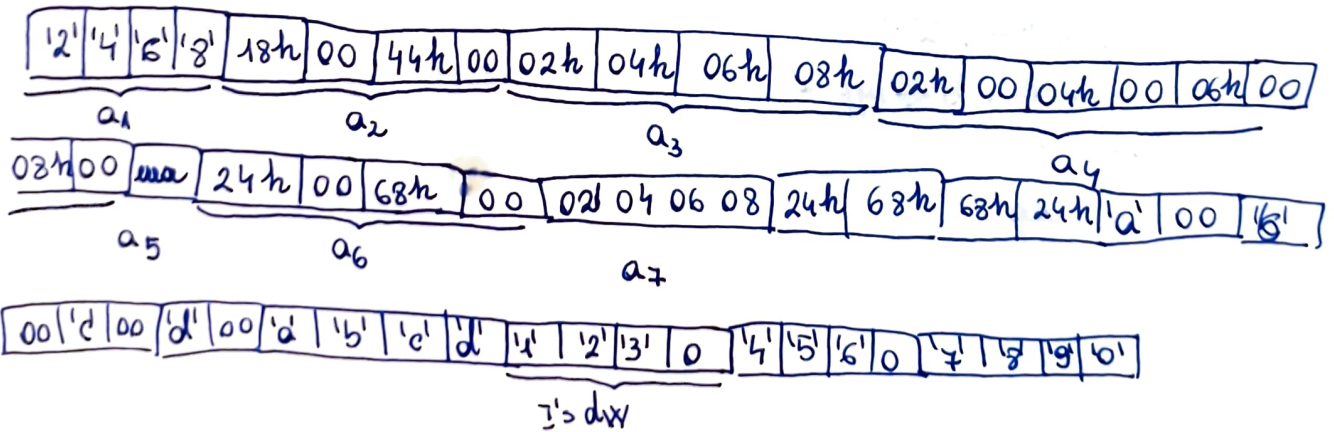
also div a', b', c', d'

all dw 'abcd'

a12 dd '123', '456', '4890'


$$68:16 = 4 \text{ R } 4$$

64

$$4:16 = 0.25$$


String instructions

1. Being given a string of bytes containing lowercase letters, build a new string of bytes containing the corresponding uppercase letters.

DS:

a db 'a', 'b', 'c'

la equ \$ - a

~~b db 'a'~~

b times la db 0

CS:

mov ECX, la

mov ESI, a

mov EDI, b

cld → clear direction flag

jecxz end-loop

loop:

~~STOSB AL~~

LODSB AL ← 'a'

SUB AL, 'a' - 'A'

STOSB i b ← upp 'a'

loop lup

end-loop:

LODSB

cmp DL, AL

jle not

mov EDI, ESI

sub EDI, 2

STOSB

mov AL, DL

STOSB

mov BL, 1

not:

dec ESI

loop for

end for

cmp BL, 0

jne while

DL = d(i)

AL = d(i+1)



```
mov ECX, la
mov ESI, a
```

```
// Std ; set direction flag DF=0
```

```
cld ; DF=1
```

```
mov EDI, d
```

```
JECXZ end-loop
```

~~loop~~ loop-repeta: ; low bytes of the words from the first string (in AL)

```
LODSW ; AL
```

```
STOSB ; AL
```

```
end-loop:
```

```
mov ECX, lb
```

```
mov ESI, b
```

```
cld
```

```
JECXZ End
```

~~loop~~ concatenate:

```
LODSW
```

```
mov AL, AH
```

```
STOSB
```

```
loop concatenate
```

```
End
```

; setting part

while:

```
mov BL, 0
```

3 5
5 6 4 4

BL = 01

```
mov ESI, d
```

```
mov ECX, ld-1
```

```
cld
```

```
JECXZ endwhile endfor
```

~~while~~ for:

```
cmp 0 05B ; AL = d[i]
```

```
mov DL, AL
```

DS:

a db 'abcdef'

len equ \$-a

resb len

CS:

mov ecx, len

mov esi, a

mov edi, b + len - 1

JECXZ end.

loop:

cld ; DF=0

LODSB ; AL = 'a'

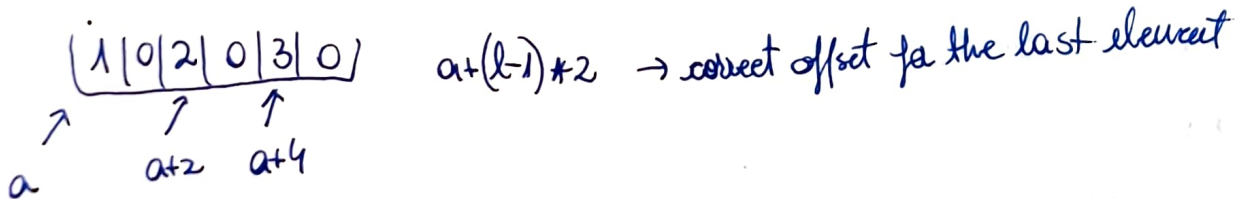
std

STOSB ; CDS:EDI, a[edi-1] = 'a'

loop

end

a dw 1, 2, 3



Ex3. 2 strings of words are given. concatenate the strings of low bytes

a dw 1, 2, 3, 4, 5

b dw 6, 7, 8, 9, 10

la equ (\$-a)/2

lb equ (\$-b)/2

c times lodsb

ld equ la*lb

Quiz 1: 10

Quiz 2: 9

Quiz: 10

1 0000 00

1 0000 000

1 0000 0000

$\frac{16}{8} = 2$
 $\frac{128}{4}$

CF=1

CD AB DC FE

$$3 = 011 \Rightarrow -3 = 101$$

$$4 = 010 \Rightarrow -4 = 110$$

$$\boxed{100} (-3-4)$$

1111111100

1234 dd 2, 16 db

34 12 34 12 16

34 12 00 00 34 12 00 00 16

2 00 00 00 2 00 00 00 2 00 00 00 20000000

00 00

15 14 13 12 11 10 9 8 AH 7 6 5 4 AL 3 2 1 0

7 6 5 AH 3 2 1 0 AL 7 6 5 4 3 2 1 0

3c 2b 1a 00 of 4d 00 00 27 5d 6e 00

$\overrightarrow{DCAB}$ DC FE

010

110

254
128

$256 \cdot 7.16^3$

0001 0000

0000 0001

16
6
96³

0110 0010

0010 0000

34 12 00 0.0 34 12 00 00 16 01 00 00 10

1 2 3 10 20 30

20

0110 0010

1100 1000

10 3 h = ~~1016~~ 3 10000 1010
21 0
16² +

103 h = 2563

200 = $\underbrace{2^7}_{128} + \underbrace{2^6}_{64} + 2^3$
192

CD AB DC FE

$$\frac{16}{6}^3 \quad \frac{200}{98} \frac{102}{102}$$

$$\begin{array}{r} 0110 \quad 0010 \\ 1101 \quad 0010 \\ \hline 100 \quad 000 \end{array}$$

$$\begin{array}{r} 182 \\ \hline 128 \quad 64 \quad 16 \\ 2^7 \quad 2^6 \quad 2^4 \quad 2^1 \end{array}$$

ax, 400h 1024
bl, ofeh
idiv bl

$$\begin{array}{r} 14.1 \\ 15 \\ \hline 16 \quad 3 \\ 90 \\ 15 \\ \hline 24 \quad 0 \\ 14 \\ \hline 254 \end{array}$$

$$\begin{array}{r} 0100 \quad 0000 \quad 0000 \\ \hline 1111 \quad 1111 \\ 0000 \quad 1111 \quad 1110 \\ \hline \end{array}$$

$$\begin{array}{r} 240 \\ \hline 224 \\ \hline 192 \\ \hline 250 = 128 + 64 + 32 + 16 + 8 + 2 \\ 2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^1 \end{array}$$

10

$$\begin{array}{r} 11111010 \\ \cdot \quad 01111 \end{array}$$

• a344h 30h 11223344h 46ab 89eh
44a3

44 a3 00 00 30 00 00 00 44 33 22 11
9c b8 6a 40

1 1 1

1 1 0

CF=1

1 0 0

CF=1

0 0 0

6553

9

16
9
144 5

256
9
2304 5

256
16
1506 3
256
406 6
2304
6370
144

6514
9

192 224 240 248 252

$$255 = 128 + 64 + 32 + 16 + 8 + 4 +$$

1 1 1 1 1 1 1 1 1 1

1 1 1 1 1 1 1 1 1 1

11 000 0000

La operațiile OR, AND, XOR, TEST, SHL, SHR, ROL, ROR,

RCL, RCR Nici o DATA const nu e pe prima pozitivă

RCL : $CF = 1$ (ca la rol + scade CF micuște și-l poți reține)

mov al, binary-num

mov cl, 2

rcl al, cl . binary-num CF

RCR

ROL/ROR : ! the last rotated bit is kept in CF

! mov → uses the little indian representation

$$12 \overline{3 \quad 10} 20 \quad 30$$

$$\begin{array}{r} 256 \\ 3 \\ \hline 768 \end{array}$$

$$310 = 16 + 768 =$$

$$103 = 2563$$

~~200~~

100h

mov al, -2

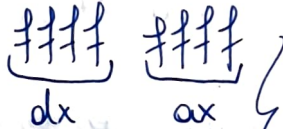
mov bl, -128

$$(-2) * (-128) = \underbrace{256}_{ax}$$

ABCDh FEDCh
← ←←

ax = ffffh

de ce pune f?

cwd → dx:ax : 

dx + 1 → dx = 0000

ax = ffffh

cwd → dx:ax : 0000 ffff (?)

AL = ?

a = 0ABCDh, 0FEDCh | CD AB DC FE

~~mov~~ lodsw ; AX = ABCD

lodsb ; AX = AB DC
AL

→ când lodsw ia ~~word~~-ul ca în DS
→ când lodsb ia ~~byte~~-ul din Mem. seg.

lodsw ; AX = ABCD

lodsw ; AX = FEDC ⇒ AL = DC !!!

mov al, -2

mov bl, -128

mul bl ; o să vadă al = 254⁽¹²⁸⁾ și bl = 128(+256)

ax = 7F00h

nu dă error

A B C D E F
10 11 12

16
9
144
44
188
15
16
90
515
240
1

a db 1, 2, 3

b dw 1, 2, 3

c dd 1, 2, 3

MEMORY SEGMENT

01 02 03

01 00 02 00 03 00

01 00 00 00 02 00 00 00 03 00 00 00

d dw

300, 400, 500
012Ch 0180h 01F4h

MEMORY SEGMENT

2C 01 90 01 F4 01

location counter = the number of generated bytes (\$)
 in hexa

\$\$ → the start of the current section

\$ - \$\$ → how far into the section (how many bytes)

mov dh, 62 | 62(s)

mov eh, 200 | -56(s)

sub dh, eh

$$(256 + 62) - 200 =$$

$$-34 - 4$$

$$-128, 127$$

$$\frac{253}{128} = 2$$

$$253 = \underbrace{2^7}_{128} + \underbrace{2^4}_{16} + \underbrace{2^3}_{8} + 2^1 + 2^0$$

$$252 = 128 + 16 +$$

$$\begin{array}{r} 10011011 \\ 10011010 \\ \hline 10011010 \end{array}$$

$$15 \cdot 16 + 14$$

$$\begin{array}{r} 256 \\ 256 \end{array}$$

$$\begin{array}{r} 15 \\ 16 \\ \hline 90 \quad 3 \\ 15 \\ \hline 240 \\ 264 \end{array}$$

$$16 \frac{7}{20} 2$$

$$\begin{array}{c} CH \\ 00 \boxed{0 \pm \pm \pm \pm} \\ \pm \pm \pm \pm \pm \pm \pm \pm \end{array}$$

$$\overbrace{\pm \pm \pm \pm}^{CH} = ecx$$

Seminar 6

15.12.2022

ASC

Multi module programming

① Write a ~~function~~ program that concatenates 2 strings by calling a function from another module and then prints the result on the screen.

Main:

global

segment data use 32 class = data:

```
str1 db "Hello", 0
len1 equ $-str1
str2 db "world", 0
len2 equ $-str2
strcat resb len1 + len2, 0
```

segment code use 32 class = code:

start:

```
push dword str1
push dword str2
call strcat
add ESP, 4 * 3
concatenate:
global strcat
segment code use 32 class = code:
strcat:
    mov EBX, 0 ; string result position
    mov EDI, [ESP + 4]
    mov ESI, [ESP + 12]
    cld
    push dword strcat ; the address of the
                        ; resulted string
    call [printf]
    add ESP, 4
    push dword 0, call [exit]
```

strcat:

```
mov EBX, 0 ; string result position
mov EDI, [ESP + 4]
mov ESI, [ESP + 12]
cld
```

while-loop1:

lodsb

cmp AL, 0

jz end-loop1

stosb

jmp while-loop1

end-loop1:

mov ESI, [ESP+8]; shr 2

while-loop2:

lodsb

cmp AL, 0

jz end-loop2

stosb

jmp while-loop2

end-loop2:

mov AL, 0

stosb

ret

REP MOVSB

↓

repeat an instruction
ecx times

③ (Seminar 5)

Read from the keyboard a number n in base 16 that can be repr.
~~on~~ on a word (no validation needed). Open the

data:

~~n dd 72A1h~~

n dd 0

format db "%x", 0

fd ~~dd~~ dd -1

file_name dd "in.txt", 0

acc_mode dd "r", 0

x dd 0

code:

start

push dword n

push dword format

call [scanf]

add ESP, 4*2

push dword acc_mode

push dword file_name

call [fopen]

add ESP, 4*2

mov [fd], EAX

cmp EAX, 0

je end-loop

mov ECX, 16

solve: push ECX

push dword x

push dword format-1

push dword [fd]

format-1 db "%c", 0

call [fscanf]

add ESP, 4*3

shr [n], 1

je do-not-print

push dword [x]

push dword

call [printf]

add ESP, 4*2

do-not-print:

loop-solve: $\leftarrow$ pop ECX

② Write a program that prints the sum of the digits of an unsigned number represented on a doubleword (by calling a function for the sum)

Main:

```
global start
extern sum-digits
segment data use 32 class = data
n dd 12345678
res dd 0
continuit-format db "%u", 0
segment code use 32 class = code
```

start:

push dword res

push dword [n]

call sum-digits

add ESP, 4 * 1

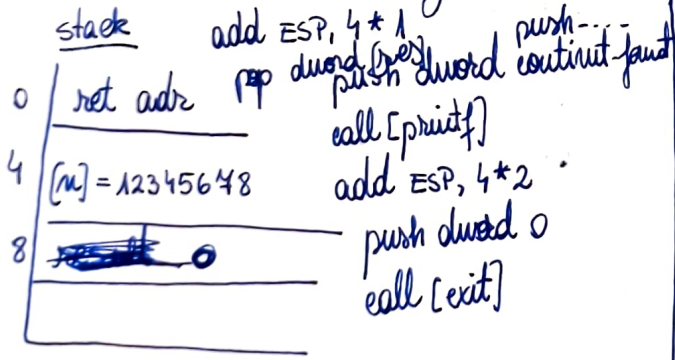
push dword [continuit-format]

call [printf]

add ESP, 4 * 2

push dword 0

call [exit]



sum-digits:

```
global sum-digits
segment data use 32 class = data
ten dd 10
segment code use 32 class = code
```

sum-digits:

mov EAX, [ESP + 4]

while-loop:

mov EDX, 0 ; EAX → EDX : EAX

div dword [ten]

add [res], EDX

mov EDX, 0

inc EAX, 1

jmp while-loop

mov [ESP + 8], [result]

mov [EDX], [result]

mov [ESP + 8], EDX

ret

12.01.2022

Exercise 1Main:data

input-format db '%s', 0

input resb 100

max dd 0

max-filename resb 100

code

start:

~~loop~~ - reading

push dword input

push dword input-format

call [scanf]

add ESP, 4*2

cmp byte [input], 'e'

jne not-end

cmp byte [input+1], 'n'

jne not-end

cmp byte [input+2], 'd'

jne not-end

cmp byte [input+3], '0'

je end

not-end:

push dword input

call most-upper

add ESP, 4

cmp dword [max], eax; ~~eax → not from function~~

jae keep-reading

mov dword [max], eax

Function

most-upper

global most-upper

data

count

filemode db 'r', 0

fd dd 0

char-format db '%c', 0

char ~~resb~~ resb 1code

most-upper: mov edx, [esp+4]

push dword filemode

push dword [ESP+4]

call [fopen]

~~add esi~~

mov dword [fd], EAX

add ESP, 4*2

keep-reading:

push dword char

push dword char-format

call [scanf]

add ESP, 4*2

cmp byte [char], -1

je done-reading

cmp byte [char], 'A'

jb keep-reading

```

cmp byte [char], 'z'
je keep-reading
inc EBX
jump keep-reading

done-reading:
mov eax, ebx
ret

```

main:

```

mov ESI, input
mov EDI, max-filename
copy-name:
movsb

inc [esi], 0
jne copy-name ← mov [edi], 0
jump keep-reading

end:
push dword max-filename
call print-file-contents
add esp, 4
call [exit]

```

```

push dword [file]
call [fclose]
add esp, 4

```

```

II function
print-file-contents
mov edx, ...
push dword filename
push dword [esp+4]
call [fopen]
mov dword [file], eax
add esp, 4*2

```

```

push dword [esp+4]
call [printf]
add esp, 4
print-loop:
push dword char
push dword format
push dword [file]
add esp, 4*3
inc [char], -1
je done
push dword char
push dword format
call [printf]
add esp, 4*2
jump print-loop

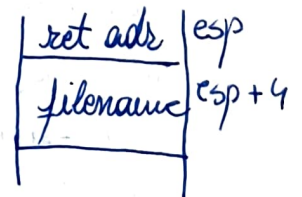
done:
ret

```

```

data
filename db "r", 0
print-format db "%c", 0
file dd 0
name-format db "%s", 0
char db 0

```



keep-reading:

push dword nr

push dword format-nr

~~call [scanf]~~

~~add esp, 4*2~~

push dword [fd]

call [scanf]

add esp, 4*3

mov eax, [nr]

shr eax, 16

mov ah, 0

push eax

call prime-nr

cup eax, 0

je ~~not~~ keep-reading

push dword [nr]

push dword format

call [printf]

add esp, 4*2

jump keep-reading

end:

push dword

*) cup eax, 0

jle end

push dword [fd]

call [fclose]

add esp, 4

function I

prime-nr

DS

ES

prime-nr:

mov ebx, [esp+4]

mov bl, 2

mov ecx, 0

division:

div ~~to~~ bl

cup ah, 0

~~je~~ je end

mov eax, [esp+4]

inc bl

cup bl, al

je end-yes

jump division

end-yes:

mov el, 1

end-not:

mov eax, ecx

ret

Exercise 2

char db, 0
format db '%c', 0

(*)
push dword [char] → char is a byte ⇒ conversion
push dword format
call [printf]
add esp, 4 * 2

(*)
mov eax, 0
mov al, [char]

Exercise 2

DS
~~file_name~~ format db "%s", 0
file_mode db "r", 0
fd dd -1
file_name resb 100
format_wx db "%d", 0
wx dd 0

CS

start:

push dword file_mode
push dword file_name
call [scanf]
add esp, 4 * 2

push dword file_mode
push dword file_name
call [fopen]
add esp, 4 * 2